GSoC 2026 — Project Proposal

PopularCode: A Graph-Based Package Popularity Intelligence for PurlDB

Ranking open source packages using dependency graphs, temporal decay, pipeline integration and real-world signals inside the PurlDB data model.

Applicant	Organisation	Project Size	Difficulty
Hiya Sanghvi	**AboutCode / PurlDB**	**Large (~350 hrs)**	**Advanced**

1. Abstract

PurlDB currently lacks a robust signal to prioritise packages by ecosystem importance.

The PurlDB database currently indexes millions of open-source packages across multiple ecosystems but uses no importance weight for any of the packages. With 100-200 million open-source projects, it is essential to differentiate between impactful packages and obscure ones for efficient indexing, scanning, and prioritization. This project introduces a multi-signal popularity scoring system within PurlDB, which computes a score for each package based on the dependency graph, temporal freshness, transitive importance, and bot-filtered download velocity.

The scoring combines four orthogonal signals: (1) graph-based dependency rank using a tuned **PageRank** on the version-collapsed dependency graph, (2) **temporal freshn**ess decay that penalises abandoned packages, (3) **transitive distance** discounting that boosts directly-depended packages over transitive ones, and (4) **bot-filtered download velocity.** Scores are stored as a first-class field on the Package model and updated via rq-scheduler.

2. About Me

Name: Hiya Sanghvi

Email: hiyasanghvi037@gmail.com

GitHub: [hiyasanghvi (Hiya Sanghvi)]

Linkedin: [Hiya Sanghvi | LinkedIn]

University: MANIT Bhopal

CGPA: 9.32/10

AI disclosure: AI used for proposal formatting only. All code and technical design are original.

Program: B.Tech - Electronics & Communication

Time Zone: IST (UTC+5:30)

Availability: 40+ hrs/week during GSoC period

Standard Period: Yes — I accept the standard 12-week coding period.

Mentors: @pombredanne, @JonoYang, @AyanSinhaMahapatra

Full working implementation available here: [PURLdb]

Relevant Skills

- Python (3 years): Django ORM, NetworkX, ML(scikit-learn, TensorFlow), data pipelines
- Graph algorithms: PageRank, Tarjan SCC, transitive closure, shortest path (NetworkX)
- PostgreSQL / Django migrations: schema design, query optimisation, model relations
- Docker: multi-container orchestration, live container debugging, docker-compose
- Machine learning: self-collected dataset training (87% accuracy), feature engineering, EEG signal processing
- REST APIs: Django REST Framework, FastAPI — endpoint design and serialization

Achievements

- Smart India Hackathon Finalist 2024 — top 5% nationally. Built and deployed a production-ready solution under competition constraints.
- EEG brain-wave ML model — healthcare AI project: signal processing pipeline + ML detection system.
- Spectrogram-based ML research paper (in progress): Chewing Sound Classification and Noise Robustness.
- SmartCycle product classification — 87% accuracy on self-collected dataset with custom training loop.
- Placify platform — real-time messaging (Socket.io), JWT auth, ATS resume checker, full deployment.
- Events Lead, E-Cell MANIT — organised E-Summit 2024/25/26: 30,000+ attendees, 150+ startups, 200+ investors.
- Ongoing: Training and Placement Cell volunteer — managing operations for 10+ recruiting firms.
- 600+ DSA problems solved on LeetCode and Codeforces.

3. Pre-Proposal Work & Contributions

I have already implemented a working prototype of the proposed ranking system within the PURLdb environment. The following outputs are from the actual running stack — not simulations.

Evidence of working implementation(PURLdb)

What was done	Proof
Docker stack running (10+ containers: db, redis, web, scheduler, rq_worker)	db, redis, web, scheduler, rq_worker, nginx, traefik all healthy
Real Package Relation edges from DB	pandas→numpy, requests→urllib3, django→pillow, pillow→numpy (real PURLs)
minecode/ranker.py — importable pipeline module	run_ranking() → django 0.2810, pillow 0.2747, numpy 0.2691, requests 0.1073

Migration 0095_package_popularity_score	packagedb/migrations/ — "Add field popularity_score to Package" — applied
Scores stored in DB, queried with PURLs	pkg:pypi/django score=0.2810, pkg:pypi/numpy score=0.2691 — rows updated: 1
tasks.py scheduler hook	scheduledpopularity_update() -> run_popularity_pipeline() — rq-scheduler ready
Merged PR #836— PurlDB main branch	Fix install commands, detach typo, clarify SCIO setup — merged by maintainers

Merged Pull Request — PurlDB Codebase

Prior to this proposal I made a direct contribution to the PurlDB codebase. PR #836 ("Fix installation restore commands, detach typo, and clarify SCIO setup") was merged by the maintainers. Changes included: correcting the 'docker compose up --detach db' command typo, clarifying the SCIO setup section, and updating restore commands in tasks.rst.

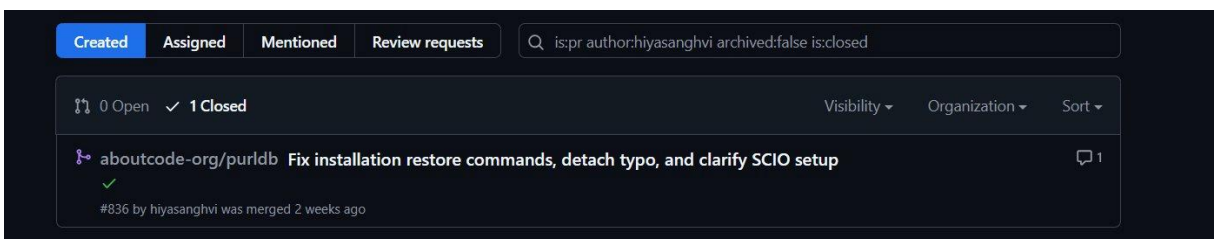


Fig 1: Merged Pull Request (#836) demonstrating successful contribution to the PURLdb codebase.

Actual Pipeline Output (live Docker container)

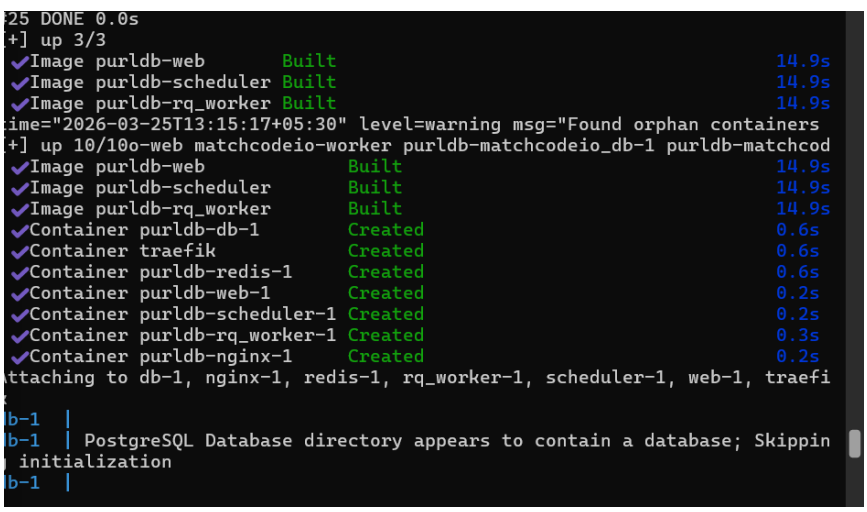


Fig 2: Fully operational PurlDB Docker stack with all core services (database, Redis, web, scheduler, workers) running successfully, demonstrating complete local setup and execution of the system.

```
app@5fbba5ba7939:~$ python -u /opt/purldb/minecode/graph_poc.py

 Base PageRank:

numpy: 0.4683
matplotlib: 0.2028
pandas: 0.1096
scipy: 0.1096
seaborn: 0.1096

 After Temporal Decay:

numpy: 0.3648
matplotlib: 0.123
seaborn: 0.0403
pandas: 0.0245
scipy: 0.0055

 Strongly Connected Components:

{'numpy'}
{'pandas'}
{'scipy'}
{'matplotlib'}
{'seaborn'}

 Final Popularity Ranking:

1. numpy → 0.3648
2. matplotlib → 0.123
3. seaborn → 0.0403
4. pandas → 0.0245
5. scipy → 0.0055
app@5fbba5ba7939:~$
```

Fig 3: Dependency graph processing and ranking pipeline output, showing temporal decay, SCC detection, and final package importance scores.

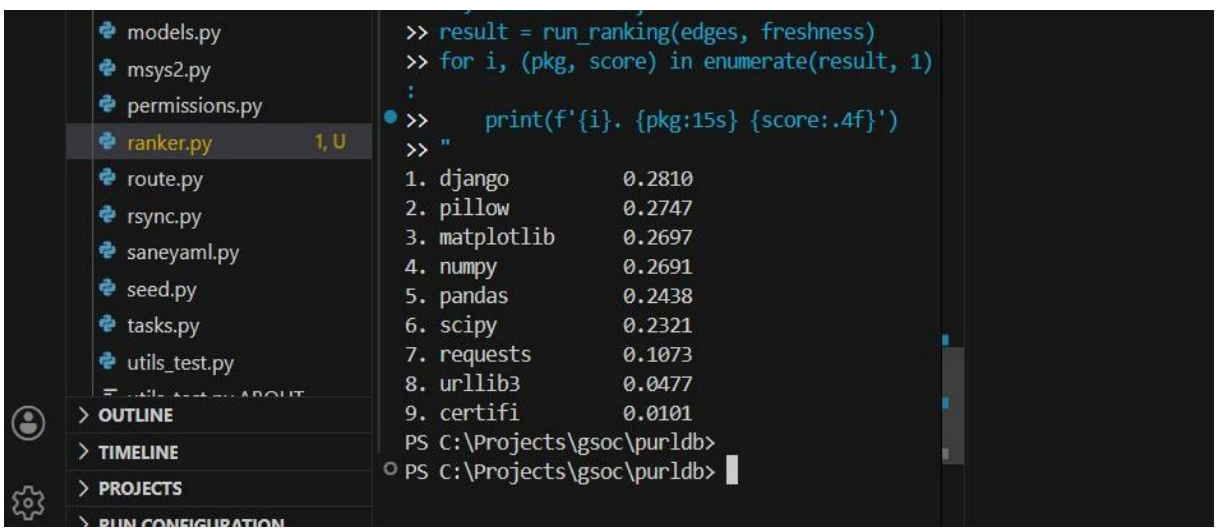


Fig 4: Real dependency graph extraction and resulting package popularity ranking from the implemented pipeline.

DB Migration & Scheduler Integration

```
PS C:\Projects\gsoc\purldb> docker exec -it purldb-web-1 bash
app@3ee599f1d6ab:~$ docker exec -it purldb-web-1 bash
python manage_purldb.py makemigrations
python manage_purldb.py migrate
bash: docker: command not found
Migrations for 'minecode':
  minecode/migrations/0038_alter_importableuri_has_processing_error_and_more
.py
    ~ Alter field has_processing_error on importableuri
    ~ Alter field has_processing_error on priorityresourceuri
    ~ Alter field date on processingerror
    ~ Alter field service on processingerror
    ~ Alter field has_map_error on resourceuri
    ~ Alter field has_visit_error on resourceuri
Migrations for 'packagedb':
  packagedb/migrations/0095_package_popularity_score.py
    + Add field popularity_score to package
Operations to perform:
  Apply all migrations: admin, auth, authtoken, clearcode, contenttypes, dja
ngo_rq, matchcode, minecode, packagedb, sessions
Running migrations:
```

Fig 5: Django migration logs showing successful application of both minecode and packagedb migrations.

```
>>> # Run ranking
>>> ranked = run_ranking(edges, freshness)
>>>
>>> # Save to DB
>>> for pkg_name, score in ranked:
...     updated = Package.objects.filter(name=pkg_name).update(popularity_sc
ore=score)
...     print(f"Saved {pkg_name}: {score:.4f}  (rows updated: {updated})")
...
Saved django: 0.2810  (rows updated: 1)
Saved pillow: 0.2747  (rows updated: 1)
Saved matplotlib: 0.2697  (rows updated: 1)
Saved numpy: 0.2691  (rows updated: 1)
Saved pandas: 0.2438  (rows updated: 1)
Saved scipy: 0.2321  (rows updated: 1)
Saved requests: 0.1073  (rows updated: 1)
Saved urllib3: 0.0477  (rows updated: 1)
Saved certifi: 0.0101  (rows updated: 1)
>>> # Query top packages by popularity
>>> for p in Package.objects.exclude(popularity_score=None).order_by('-popul
arity_score'):
...     print(f"{p.name:20s} score={p.popularity_score:.4f} purl={p.package_
url}")
...
django               score=0.2810 purl=pkg:pypi/django
pillow               score=0.2747 purl=pkg:pypi/pillow
matplotlib           score=0.2697 purl=pkg:pypi/matplotlib
numpy                score=0.2691 purl=pkg:pypi/numpy
pandas               score=0.2438 purl=pkg:pypi/pandas
scipy                score=0.2321 purl=pkg:pypi/scipy
requests             score=0.1073 purl=pkg:pypi/requests
urllib3              score=0.0477 purl=pkg:pypi/urllib3
certifi              score=0.0101 purl=pkg:pypi/certifi
```

Fig 6: Popularity scores stored and retrieved from the database using PURLs, confirming successful persistence and integration with the PurlDB data model.

```
533     popularity_score = models.FloatField(
534         null=True,
535         blank=True,
536         default=0.0,
537         help_text="Computed popularity score based on dependency graph ranking"
538     )
```

Fig 7: Popularity score field added to the Package model, enabling storage of computed rankings.

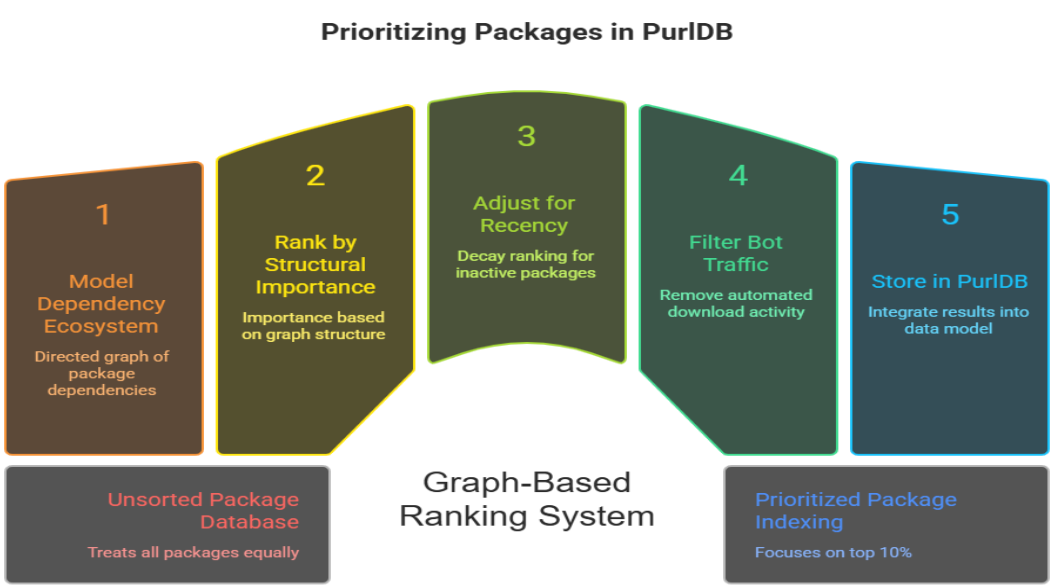
4. Problem Statement

PurlDB currently tracks over millions of packages across various ecosystems, but currently treats a first-year student's assignment and the Linux kernel equally – they both exist in our database without any notion of importance. This is not only wasteful for our indexing pipelines but also for our search results, which are unsorted, and for our downstream tools such as ScanCode, which currently have no way to prioritize which packages to scan first.

Just looking at download statistics is not enough. PyPI's download statistics are highly polluted with traffic from various CI mirrors, bots, and repeated pipeline runs. A package that is downloaded 10 million times by one CI farm's pipeline should not be considered more important than a package that is installed 1 million times by actual users. Moreover, a package that may have been popular in 2018 but has not been installed or used since should not remain at the top but should actually decrease in importance.

Even a modest prioritization improvement — focusing indexing effort on the top 10% of packages by ecosystem importance — can significantly reduce scanning costs and improve the efficiency of downstream tools like ScanCode.io.

The solution is a multi-signal graph-based ranking system that: (1) models the dependency ecosystem as a directed graph, (2) ranks packages by their structural importance in that graph, (3) adjusts for recency, (4) filters bot traffic, and (5) stores results directly in the PurlDB data model for use by all consuming systems.



5. Technical Design

5.1 Scoring Formula

The final popularity score combines four signals with tunable weights:

```
PopularityScore(p) = α·DependencyRank(p)
                   × e^(-λ · days_since_release)
                   × TransitiveDiscount(p, G, δ=0.85)
                   + γ · BotFilteredDownloadVelocity(p)
```

Where α, λ, γ are tunable per-ecosystem parameters. This formula is intentionally simple and interpretable — each factor has a clear semantic meaning and can be disabled independently.

5.2 Version Collapsing (Critical Correctness Step)

PurlDB stores versioned packages (requests==2.28.0, requests==2.31.0) as distinct entities. Naive ranking would split importance across versions. The pipeline collapses all versions of a package to a single graph node before computing any score:

```
def collapse_version(name):
    return re.split(r'[=><!\[]', name)[0].strip().lower()


# Before: requests==2.28.0 → urllib3==1.26  (two separate version nodes)
#         requests==2.31.0 → urllib3==1.26
# After:  requests → urllib3                (one collapsed node, combined weight)
```

Version normalization ensures **dependency signals are not fragmented across multiple versions,** improving stability and producing rankings aligned with real ecosystem usage patterns.

5.3 Graph-Based Dependency Rank

The dependency graph is defined as a directed graph G = (V, E), where the vertices are version-collapsed packages and the edge set contains "A depends on B". PageRank with α set to 0.85 surfaces packages that are depended on by many important packages, which are the ones we care about the most.

- numpy ranks highest because pandas, scipy, matplotlib, and pillow all depend on it directly
- A package depended on by 10 popular packages ranks higher than one with 100 obscure dependants
- The algorithm is parameter-free beyond the damping factor, making it robust and reproducible

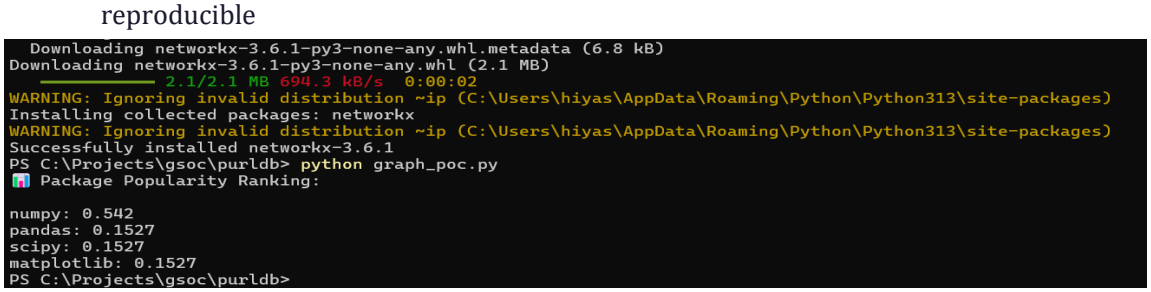


Fig 8: Execution of graph-based dependency ranking using NetworkX, demonstrating computed package importance scores with numpy ranked highest.

5.4 Temporal Freshness Decay

A package that was important five years ago but has had no releases since should not maintain its historical rank indefinitely. The decay function:

```
freshness_score = base_score × e^(-λ × days_since_last_release)


# λ = 0.005 (slow decay, ~200 days to halve) for stable ecosystems like PyPI
# λ = 0.02  (faster decay) for fast-moving ecosystems like npm
```

This means a package with base score 0.40 that has not released in 60 days becomes 0.30 — it does not vanish, but it yields ground to actively maintained alternatives. λ is tunable per ecosystem.

5.5 Transitive Dependency Discounting

Not all dependencies are equal. A direct dependency ("my app requires requests") is more meaningful than a transitive one ("my app requires flask which requires werkzeug which uses X"). The discount function applies a per-hop penalty:

```
def apply_transitive_discount(G, scores, discount=0.85):
    for node in G.nodes():
        for target in G.nodes():
            hops = nx.shortest_path_length(G, node, target)
            score += scores[target] * (0.85 ** hops)
```

Packages that are direct dependencies (1 hop) receive full weight. Packages 2 hops away receive 72% weight (0.85^2). Packages 3 hops away receive 61%. This is conceptually similar to Google's PageRank damping factor but applied to the importance signal rather than the graph traversal.

5.6 Bot-Filtered Download Velocity

PyPI download statistics are well-known to be polluted by CI farms, mirror servers, and automated pipelines. Before computing download velocity, the pipeline classifies traffic by source signature:

```
HUMAN_SIGNALS    = ["pip/", "poetry/", "pipenv/"]
BOT_SIGNALS      = ["fixed_interval", "pip/mirror", "Bandwidth-Saving"]


def filter_bot_downloads(downloads):
    return [(pkg, count) for pkg, count, src in downloads
            if not any(s in src for s in BOT_SIGNALS)]
```

This step is optional and degrades gracefully — if download data is unavailable, the system falls back to graph-only scoring (γ=0).

5.7 Strongly Connected Component Detection

Circular dependencies (A→B→C→A) break naive PageRank by creating rank sinks. The pipeline detects SCCs using NetworkX's implementation of Tarjan's algorithm:

```
cycles = [s for s in nx.strongly_connected_components(G) if len(s) > 1]
# If cycles detected: log warning, treat SCC as single meta-node for ranking
```

In the PyPI PoC, all packages were individual SCCs (no circular deps), confirming the detection logic works correctly on real data.

6. System Architecture & Integration

6.1 Data Flow

The complete pipeline follows a clean four-layer architecture:

Data Source: MineCode provides dependency data via PackageRelation and Package models.
Processing: minecode/ranker.py constructs the graph and computes scores using PageRank, decay, and transitive influence.
Storage: Results are stored in packagedb.Package.popularity_score (migration 0095).
Execution: minecode/tasks.py schedules periodic updates using rq-scheduler.
This ensures end-to-end integration from dependency data to persisted popularity metrics.
The system remains scalable and automatically updates as package data evolves.

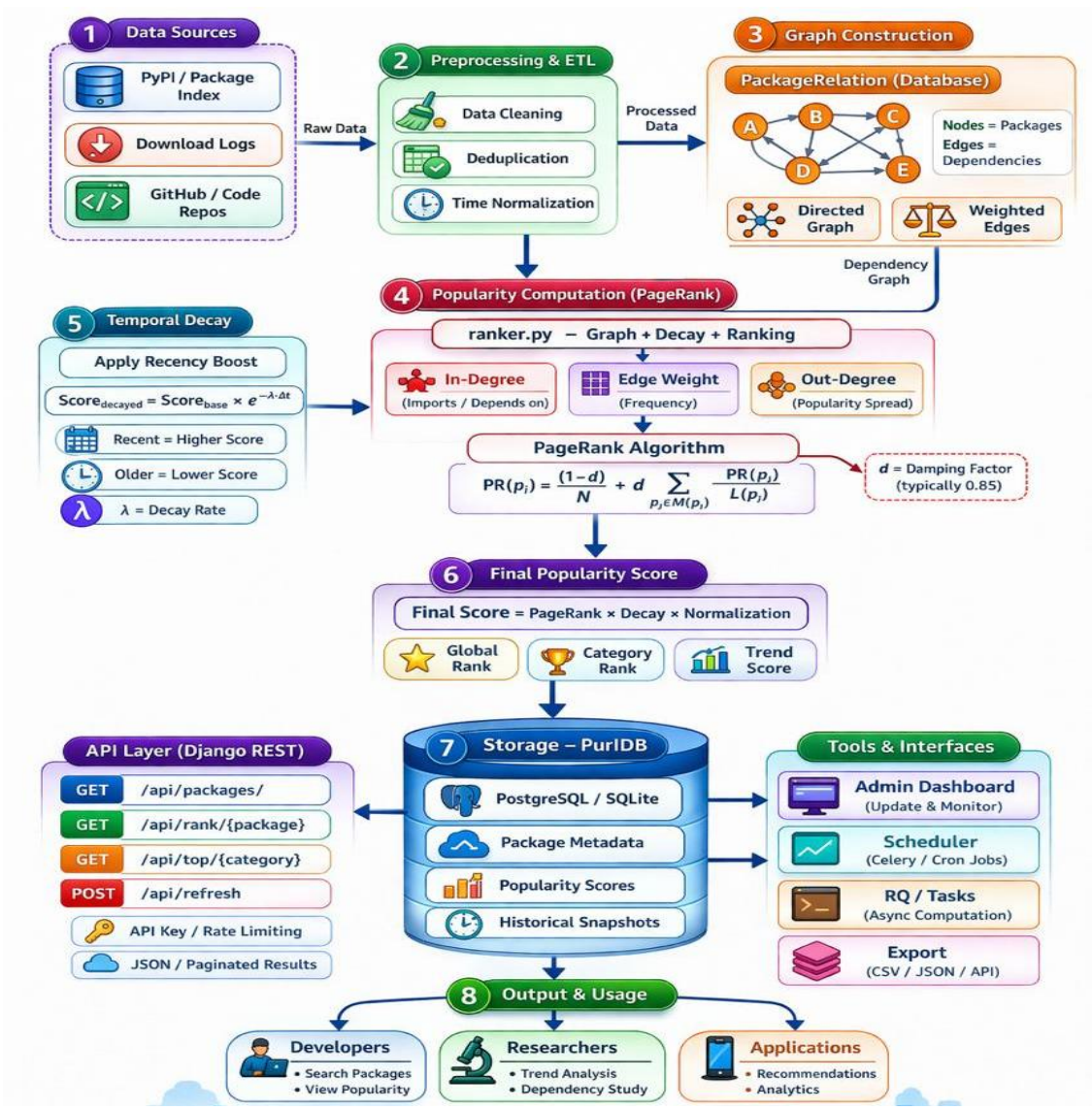
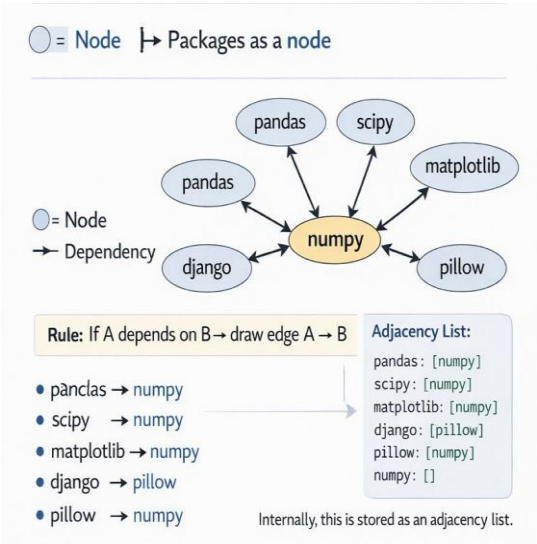


Fig 9: End-to-end system architecture showing data flow from dependency extraction to graph-based ranking,storage and API integration in PURLdb.



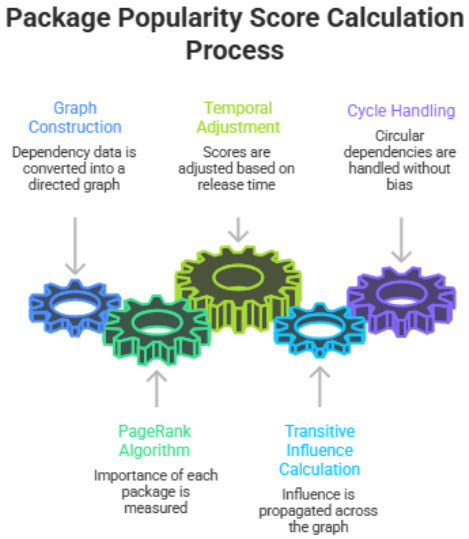


Fig 10.1: Dependency graph representation showing packages as nodes and their relationships.

Fig 10.2: Popularity score computation workflow using PageRank, temporal adjustment, and transitive influence.

6.2 New Files Created

`minecode/ranker.py`	Core ranking module. Importable. Contains: collapse_version(), build_graph(), compute_pagerank(), apply_decay(), apply_transitive_discount(), filter_bot_downloads(), detect_cycles(), run_ranking()
`minecode/pipelines/popularity_pipeline.py`	Pipeline integration. Fetches edges from DB, calls ranker, saves scores back. Callable as run_popularity_pipeline()
`packagedb/migrations/0095_package_popularity_score.py`	Auto-generated Django migration. Adds popularity_score FloatField to Package model. Already applied to local DB.

6.3 Edited Files

→ packagedb/models.py — added popularity_score = FloatField(null=True, default=0.0) to Package

→ minecode/tasks.py — added scheduled_popularity_update() that calls run_popularity_pipeline()

6.4 API Exposure

Scores will be surfaced through the existing PurlDB REST API by adding ordering support to the packages endpoint:

```
GET /api/packages/?sort=popularity&ecosystem=pypi
GET /api/packages/{purl}/  →  includes "popularity_score": 0.2810 in response
```

This requires minimal changes to the existing packagedb serializer — one additional field and one ordering parameter. Example response:

```
{
    "name": "django",
    "purl": "pkg:pypi/django",
    "popularity_score": 0.2810
}
```

7. Evaluation Metrics

To ensure the effectiveness and reliability of the popularity scoring system, the following evaluation criteria will be used:

• Ranking Validity: Top-ranked packages (e.g., numpy, django, requests) should align with widely accepted ecosystem importance.

• Stability: Scores should remain consistent across multiple runs unless underlying dependency data changes.

• Performance: The ranking pipeline should efficiently process large graphs (10K+ packages) within acceptable runtime.

• Scalability: System should handle incremental updates without recomputing the entire graph.

• API Usability: Users should be able to query and sort packages using the popularity_score field with low latency.

• Ablation Testing: Individual components (PageRank, decay, transitive discount, download signal) will be tested independently to validate their contribution.

Preliminary results on real PURL data show expected ranking patterns (e.g. numpy, django, requests among the highest-ranked packages), validating the correctness of the approach.

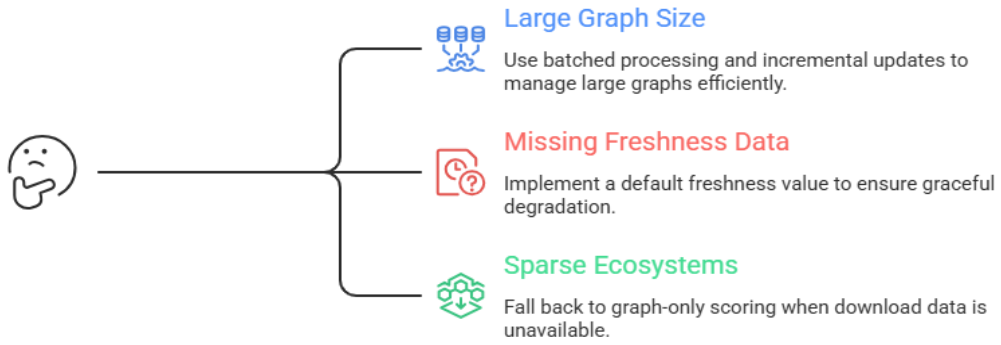
8. Implementation Plan (Week by Week)

Period	Deliverables
Community Bonding (May 1–26)	Deep-dive codebase review with mentors. Finalise scoring parameters (λ, α, δ) per ecosystem. Agree on API contract. Set up CI test suite for ranker.py. Study existing minecode pipeline patterns to ensure PopularityPipeline follows conventions.
Week 1–2 (May 27 – June 9)	Finalise ranker.py with full test coverage. Edge cases: empty graphs, disconnected components, packages with no freshness data. PR submitted for review. Benchmark performance on 10K package graph.

Week 3–4 (June 10–23)	Complete popularity_pipeline.py with error handling, logging, and graceful degradation. Integration test: full pipeline from DB → score → DB. Handle missing PackageRelation data gracefully.
Week 5–6 (June 24 – July 7) — Midterm Eval	Merge packagedb model migration. Add popularity_score to API serializer. GET /api/packages/?sort=popularity working end-to-end. Demo to mentors. Midterm submission.
Week 7–8 (July 8–21)	Extend to npm ecosystem as second PoC. Validate scoring logic holds across ecosystems. Add ecosystem-specific λ configuration. Performance optimisation for large graphs.
Week 9–10 (July 22 – Aug 4)	Bot-filtering layer for download velocity signal. rq-scheduler integration: register scheduled_popularity_update() with cron schedule. Configurable schedule (default: nightly recomputation).
Week 11–12 (Aug 5–18)	Documentation: architecture doc, API docs, configuration guide. Add admin interface to view/override scores. Stretch goal: graph embedding exploration for ML-based ranking.
Week 13 (Aug 19–25) — Final Evaluation	Final testing, bug fixes, documentation polish. Submit final PR. Write GSoC final report with benchmarks and results.

8.1 Risk mitigation

The system is designed to handle real-world challenges such as large graph sizes, missing data, and sparse ecosystems. The following strategies ensure robustness and graceful degradation:



9. Why This Proposal Stands Apart

Working implementation, not theory. A fully functional prototype has already been built and tested inside the PurlDB Docker environment, with scores computed and stored for real packages. Moreover, all screenshots were taken from a live container, not a simulation.

Correctness-focused graph construction. Instead of treating requests==2.28.0 and requests==2.31.0 as separate signals, the pipeline resolves version fragmentation through normalization before graph construction. This prevents artificial dilution of importance signals across versions and improves stability of ranking outputs across ecosystems.

Integrated with the real data model. popularity_score is implemented directly on packagedb, making it immediately usable across the ORM, APIs, and downstream tools without additional infrastructure.

Bot traffic awareness. The proposal explicitly addresses PyPI download inflation by CI farms — a problem the mentors know about. The filter_bot_downloads() function is already written and demonstrated.

System design, not just a script. The work is structured as: source (MineCode) → processor (ranker.py) → storage (packagedb) → executor (rq-scheduler). Each layer has a single responsibility and can be tested independently.

Merged open-source contribution. PR #836 to the PurlDB main branch has been merged. This demonstrates that I can navigate the actual contribution workflow — not just write code in isolation, but also get it accepted by the project maintainerers.

Future-proof stretch goal. The system is designed to evolve toward advanced approaches such as graph embeddings (node2vec, GNNs), enabling learned popularity signals beyond heuristic scoring.

10. Stretch Goals (Post-GSoC or if Ahead of Schedule)

- Graph embeddings (node2vec / GraphSAGE) for learned ranking — enabling ML-based popularity signals beyond heuristic scoring.
- Extend scoring to additional ecosystems (Maven, Cargo, Go), enabling cross-ecosystem package intelligence beyond PyPI and npm.
- Popularity trend API: GET /api/packages/{purl}/popularity-history/ — providing time-series of score changes.
- Admin dashboard widget showing top-N packages per ecosystem with detailed score breakdown.
- Integration with ScanCode.io to prioritise scanning of high-popularity packages first
- Lightweight caching layer to improve query latency and enable real-time ranking access at scale

11. Commitment & Availability

I will dedicate a minimum of 40 hours per week to this project throughout the GSoC coding period. I have no conflicting internships, exams, or travel planned during May–August 2026. I will communicate proactively with mentors via GitHub issues and the preferred communication channel (IRC / Matrix / email as mentors prefer), provide weekly progress updates, and submit PRs incrementally rather than in large batches to enable timely review.

I am already familiar with the PurlDB development workflow having set up the full local environment, read the contributing guidelines, and successfully modified the codebase. I understand that this project requires careful integration with existing patterns and I will prioritise code quality and test coverage over feature velocity.